



DISEÑO Y REALIZACIÓN DE PRUEBAS

ENTORNOS DE DESARROLLO



ÍNDICE

Planificación de pruebas

Estrategia de pruebas

Técnicas en el diseño de casos de prueba

Documentación de pruebas

Pruebas de código

Depuración

Pruebas unitarias Junit

Cobertura de código

PLANIFICACIÓN DE LAS PRUEBAS

- Durante todo el proceso de desarrollo de software, desde la fase de diseño, en la implementación y una vez desarrollada la aplicación, es necesario realizar un conjunto de pruebas que permitan verificar que el software que se está creando, es correcto y cumple con las especificaciones impuesta por el usuario.

ESTRATEGIAS DE PRUEBAS DE SOFTWARE

- Para llevar a cabo el proceso de pruebas, de manera eficiente, es necesario implementar una estrategia de pruebas a lo largo de toda la duración del proyecto:
 1. Las pruebas empezarían con las **pruebas unitarias** realizadas por cada desarrollador en el código implementado.
 2. Después estarían las **pruebas de integración**, donde se prestan atención al diseño y la construcción de la arquitectura del software. Se toman módulos probados mediante pruebas unitarias.
 3. El siguiente paso sería **las pruebas de validación**, donde se comprueba que el sistema construido cumple con lo establecido en el análisis de requisitos de software. Interviene el cliente.
 4. Finalmente, **pruebas de sistema** donde se verifica el funcionamiento total del software y otros elementos del sistema. Se prueba el software como un todo.

PRUEBAS UNITARIAS

Se busca la eliminación de errores en la lógica interna. Se realizan pruebas sobre:

- Interfaz del módulo, para asegurarnos de que la información fluye adecuadamente.
- Estructuras de datos, para asegurar su integridad.
- Condiciones límite, para asegurar que funciona correctamente en límites establecidos por los procesos.
- Todos los caminos aceptados, para asegurarnos que todas las instrucciones se ejecutan al menos una vez.
- Todos los caminos de manejo de errores.

PRUEBAS DE INTEGRACIÓN

Se observa como interaccionan los distintos módulos. Existen dos enfoques para llevarlas a cabo:

- **Integración no incremental:** Se prueba cada módulo por separado y luego se combinan todos de una vez, se prueba todo el programa completo.
- **Integración incremental:** El programa completo se va construyendo y probando en pequeños segmentos, en este caso los errores son más fáciles de localizar. Se dan dos estrategias: ascendente y descendente.

PRUEBAS DE VALIDACIÓN

- **Prueba alfa:** se lleva a cabo por el cliente en el lugar de desarrollo. El cliente utiliza el software bajo supervisión del desarrollador que irá recogiendo los errores y problemas de uso.
- **Prueba beta:** se lleva a cabo por los usuarios finales en el lugar de trabajo. El desarrollador no está presente. El usuario registrará los errores que vayan ocurriendo y se los hará llegar a los desarrolladores.

PRUEBAS DE SISTEMA

El objetivo de estos tipos de pruebas es ejercitar profundamente el software, llegando a sus límites y comprobando que su funcionamiento es el esperado.

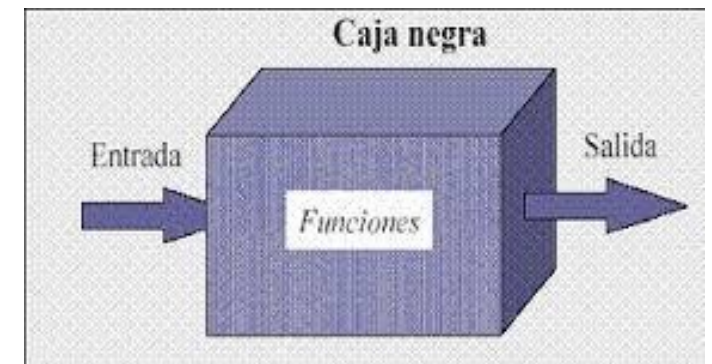
- Pruebas de recuperación: se fuerza el fallo del software y se verifica la recuperación del mismo.
- Prueba de seguridad: verificar que el software está protegido contra accesos ilegales y comportamientos sospechosos.
- Prueba de resistencia (Stress): enfrenta al sistema a situaciones de gran demanda de trabajo, por lo que los recursos estarán trabajando al máximo.

TÉCNICAS DE CASOS DE PRUEBA

Dos enfoques fundamentales:

Prueba de la Caja Negra (Black Box Testing):

- Lo fundamental es comprobar que los resultados de la ejecución de la aplicación, son los esperados, en función de las entradas que recibe.
- Se lleva a cabo sin tener que conocer ni la estructura, ni el funcionamiento interno del sistema.
- Cuando se realiza este tipo de pruebas, solo se conocen las entradas adecuadas que deberá recibir la aplicación, así como las salidas que les correspondan, pero no se conoce el proceso mediante el cual la aplicación obtiene esos resultados.



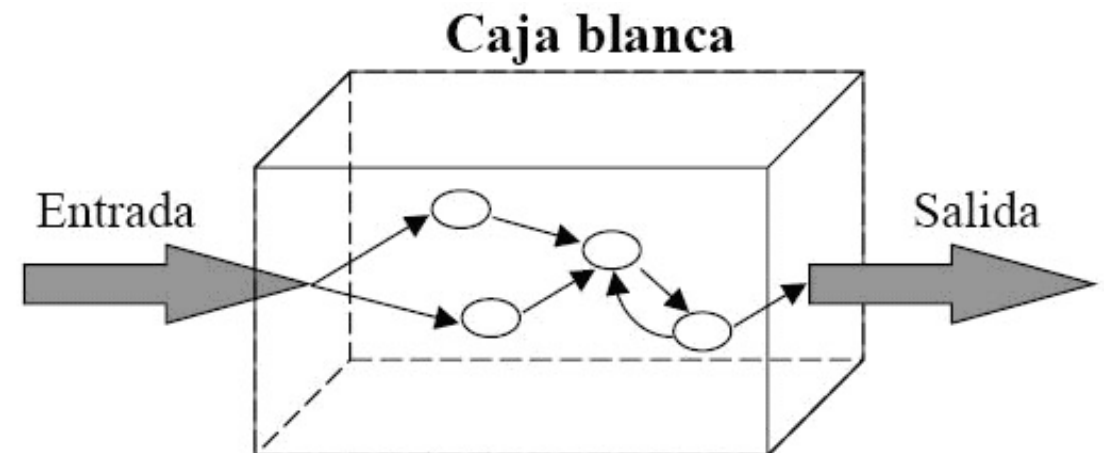
TÉCNICAS DE CASOS DE PRUEBA: PRUEBAS DE CAJA NEGRA

- También llamadas funcionales.
 - Se trata de probar, si las salidas que devuelve la aplicación, o parte de ella, son las esperadas, en función de los parámetros de entrada que le pasemos
 - No nos interesa la implementación del software, solo si realiza las funciones que se esperan de él.
 - Las pruebas funcionales intentarían responder a las preguntas ¿puede el usuario hacer esto? o ¿funciona esta utilidad de la aplicación?
-
- Ejemplo:
 - *Si estamos implementando una aplicación que realiza un determinado cálculo científico, en el enfoque de las pruebas funcionales, solo nos interesa verificar que ante una determinada entrada a ese programa el resultado de la ejecución del mismo devuelve como resultado los datos esperados.*
 - *No se consideraría, en ningún caso, el código desarrollado, ni su eficiencia, ni el algoritmo empleado, ni si hay parte de código innecesarias,etc.*

TÉCNICAS DE CASOS DE PRUEBA: PRUEBAS DE CAJA BLANCA

Prueba de la Caja Blanca (White Box Testing):

- En este caso, se prueba la aplicación desde dentro, usando su lógica de aplicación.
- Se analiza y prueba directamente el código de la aplicación.
- Es necesario un conocimiento específico del código, para poder analizar los resultados de las pruebas.



NORMAS DE CALIDAD

- Estándares BSI
 - BS 7925-1, Pruebas de software. Parte 1. Vocabulario.
 - BS 7925-2, Pruebas de software. Parte 2. Pruebas de los componentes software.
- Estándares IEEE de pruebas de software:
 - IEEE estándar 829, Documentación de la prueba de software.
 - IEEE estándar 1008, Pruebas de unidad
 - ISO / IEC 12207, 15289
 - La norma ISO/IEC 29119 de prueba de software, pretende unificar en una única norma, todos los estándares, de forma que proporcione vocabulario, procesos, documentación y técnicas para cubrir todo el ciclo de vida del software.

DOCUMENTACIÓN DE LAS PRUEBAS

Según el estándar IEEE 829-1998, los documentos que pueden producirse durante la fase de pruebas son los siguientes:

- **Plan de pruebas:** describe el alcance, el enfoque y el calendario de las pruebas. Qué elementos se van a probar, qué características se van a probar, quién va a realizar las pruebas y los riesgos.
- **Especificación de pruebas:** en varios documentos se recogerán los casos de prueba, las características para decir si pasa o no pasa, valores que se van a utilizar para las pruebas tanto de entrada como salida.
- **Informes de pruebas:** consiste básicamente en un registro donde se indica la prueba que se ha realizado y los resultados que se van teniendo.

PRUEBAS DE CAJA NEGRA (ESTRATEGIAS)

a) Partición equivalente: Consiste en dividir y separar los campos de entrada según el tipo de datos y las restricciones que conllevan. De esta forma se definen unas pruebas comunes dependiendo del tipo de dato, asegurando así pruebas eficaces y completas.

- Ejemplo: Campo *codigoPostal*, que no es obligatorio rellenar, pero que solo admite entradas de 5 dígitos numéricos → Habrá que valorar si está o no está presente, en caso de estar presente deberíamos confirmar que se trata de un número de 5 cifras con un rango mínimo y máximo (10000-99999) o también que la longitud sea 5 y solo sean números.

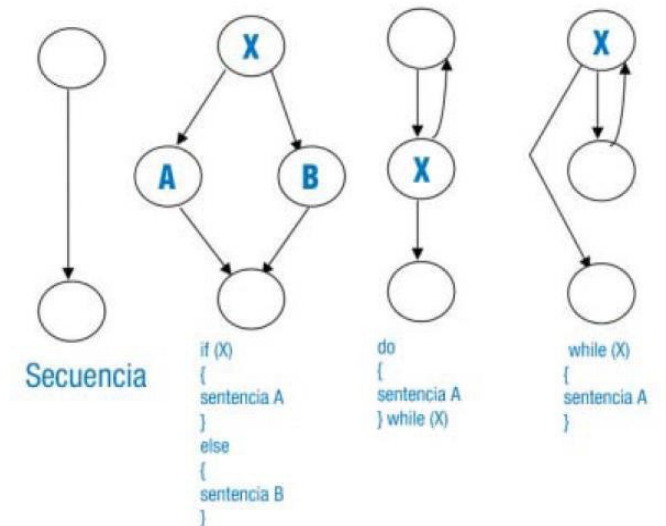
PRUEBAS DE CAJA NEGRA (ESTRATEGIAS)

b) **Análisis de valores límites:** En este caso, a la hora de implementar un caso de prueba, se van a elegir como valores de entrada, aquellos que se encuentra en el límite de las clases de equivalencia.

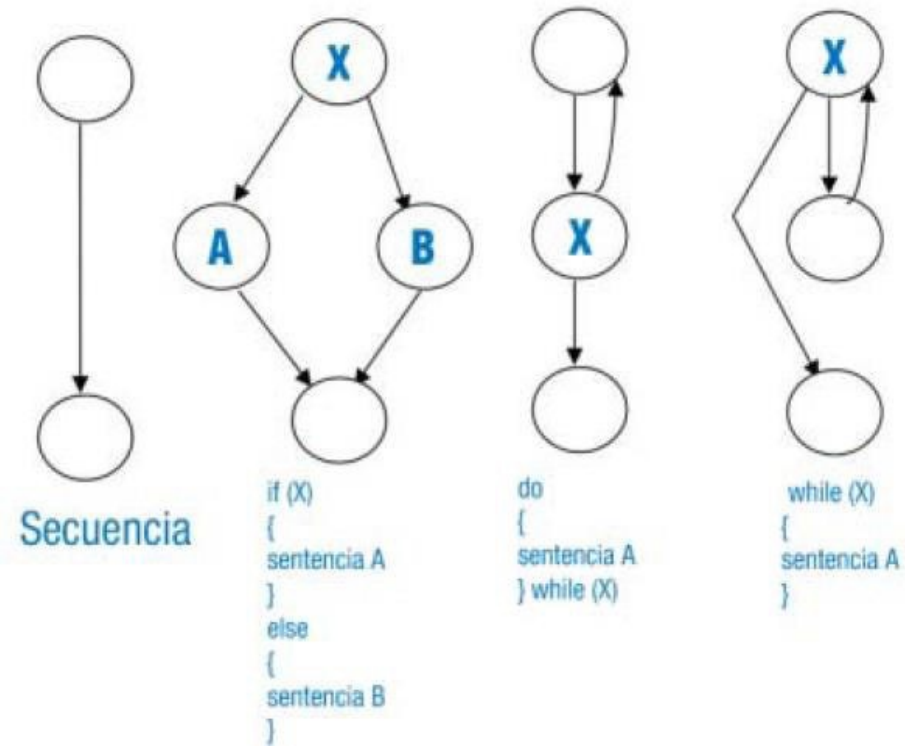
c) **Pruebas aleatorias:** Consiste en generar entradas aleatorias para la aplicación que hay que probar. Se suelen utilizar generadores de prueba, que son capaces de crear un volumen de casos de prueba al azar, con los que será alimentada la aplicación.

PRUEBAS DE CAJA BLANCA (ESTRATEGIAS)

- También llamadas estructurales.
- Para ver cómo el programa se va ejecutando, y así comprobar su correcta ejecución, se utilizan las pruebas estructurales, que se fijan en los caminos que se pueden recorrer.
- Pretenden comprobar que:
 - Se van a ejecutar todas las instrucciones del programa
 - Que no hay código no usado
 - Que los caminos lógicos del programa se van a recorrer

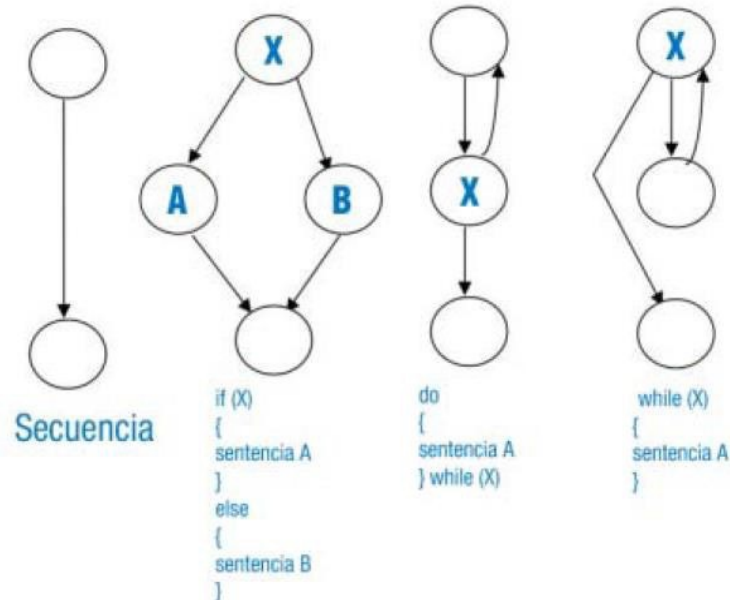


PRUEBAS DE CAJA BLANCA (ESTRATEGIAS)



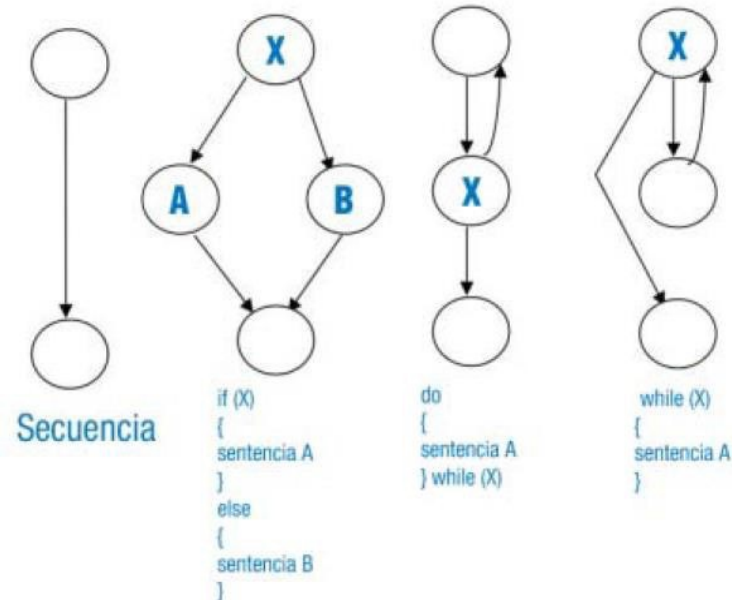
PRUEBAS DE CAJA BLANCA (ESTRATEGIAS)

- Existen diferentes criterios:
- **Cobertura de decisiones:** se trata de crear los suficientes casos de prueba para que cada opción resultado de una decisión se evalúe al menos una vez a cierto y otra a falso.



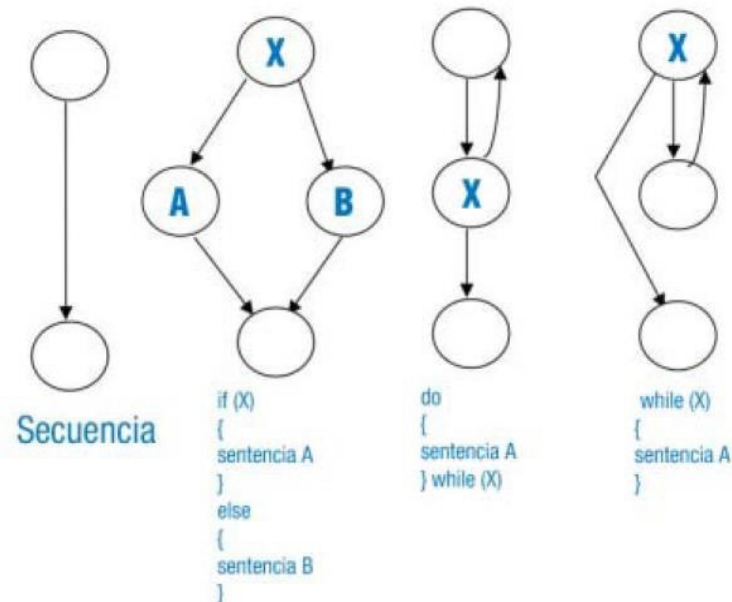
PRUEBAS DE CAJA BLANCA (ESTRATEGIAS)

- Existen diferentes criterios:
- **Cobertura de condiciones:** se trata de crear los suficientes casos de prueba para que cada elemento de una condición, se evalúe al menos una vez a falso y otra a verdadero.



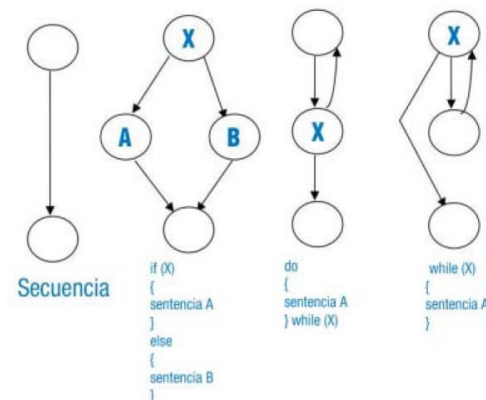
PRUEBAS DE CAJA BLANCA (ESTRATEGIAS)

- Existen diferentes criterios:
- **Cobertura de condiciones y decisiones:** consiste en cumplir simultáneamente las dos anteriores.



PRUEBAS DE CAJA BLANCA (ESTRATEGIAS)

- Existen diferentes criterios:
- **Cobertura de caminos:** es el criterio más importante. Establece que se debe ejecutar al menos una vez cada secuencia de sentencias encadenadas, desde la sentencia inicial del programa, hasta su sentencia final. La ejecución de este conjunto de sentencias, se conoce como camino. Como el número de caminos que puede tener una aplicación, puede ser muy grande, para realizar esta prueba, se reduce el número a lo que se conoce como camino prueba.



PRUEBAS DE CAJA BLANCA (ESTRATEGIAS)

- Ejemplo para comprobar que todas las funciones, sentencias, decisiones, y condiciones, se van a ejecutar:
 - Si durante la ejecución del programa, la función es llamada, al menos una vez, el cubrimiento de la función es satisfecho.
 - El **cubrimiento de sentencias** para esta función, será satisfecho si es invocada, por ejemplo como prueba(1,1), ya que en este caso, cada línea de la función se ejecuta, incluida `z=x;`

```
int prueba (int x, int y)
{
    int z=0;
    if ((x>0) && (y>0))
    {
        z=x;
    }
    return z;
}
```

PRUEBAS DE CAJA BLANCA (ESTRATEGIAS)

➤ Ejemplo (cont.):

- Si invocamos a la función con `prueba(1,1)` y `prueba(0,1)`, se satisfará el **cubrimiento de decisión**. En el primer caso, la if condición va a ser verdadera, se va a ejecutar `z=x`, pero en el segundo caso, no.
- El **cubrimiento de condición** puede satisfacerse si probamos con `prueba(1,1)`, `prueba(1,0)` y `prueba(0,0)`. En los dos primeros casos ($x < 0$) se evalúa a verdad mientras que en el tercero, se evalúa a falso. Al mismo tiempo, el primer caso hace ($y > 0$) verdad, mientras el tercero lo hace falso.

```
int prueba (int x, int y)
{
    int z=0;
    if ((x>0) && (y>0))
    {
        z=x;
    }
    return z;
}
```

PRUEBAS DE REGRESIÓN

- Durante el proceso de prueba, tendremos éxito si detectamos un posible fallo o error.
- La consecuencia directa de ese descubrimiento, supone la modificación del componente donde se ha detectado.
- Esta modificación, puede generar errores colaterales, que no existían antes.
- Como consecuencia, la modificación realizada nos obliga a repetir pruebas que hemos realizado con anterioridad → Pruebas de regresión.

PRUEBAS DE REGRESIÓN

Para ello, para que el número de pruebas no crezca demasiado:

- Se selecciona una muestra de pruebas para probar la funcionalidad del software.
- De forma más concreta, se realizan los casos de prueba asociados al las funcionalidades que se han cambiado.
- También se prueban de forma más exhaustiva los componentes involucrados.

DISEÑO DE PRUEBAS UNITARIAS (CLASES DE EQUIVALENCIA)

1. Si una condición de entrada especifica un rango → Se define una clase de equivalencia válida y dos no válidas.

Ejemplo: **la nota debe tener un valor entre 1 y 10**

2. Si una condición de entrada require un valor específico → Se define una clase de equivalencia válida y dos no válidas.

Ejemplo: **el número de departamento puede ser blanco o tener 2 dígitos.**

DISEÑO DE PRUEBAS UNITARIAS (CLASES DE EQUIVALENCIA)

3. Si una condición de entrada especifica un miembro de un conjunto $\rightarrow$ Se define una clase de equivalencia válida y una no válida.

Ejemplo: el curso puede tener los siguientes valores: "1DAM", "2DAM", "1SMR"

4. Si una condición de entrada es lógica $\rightarrow$ Se define una clase de equivalencia válida y una no válida.

Ejemplo: el salario debe ser $>$ que 0

DISEÑO DE PRUEBAS UNITARIAS (CLASES DE EQUIVALENCIA)

Condición de entrada	N.º clases válidas	N.º clases no válidas
Rango	- Contempla valores del rango.	- Un valor por encima del rango - Un valor por debajo del rango
Valor específico	- Contempla dicho valor	- Un valor por encima - Un valor por debajo
Miembro de conjunto	- Una clase por cada uno de los miembros del conjunto	- Un valor fuera del conjunto
Lógica	- Una clase que cumpla la condición	- Una clase que no cumpla la condición

DISEÑO DE PRUEBAS UNITARIAS (CLASES DE EQUIVALENCIA)

Ejercicio

Se van a diseñar las pruebas necesarias para la entrada de datos de un empleado por pantalla gráfica. Se definen tres campos de entrada, uno de ellos es una lista desplegable para elegir oficio.

1. Empleado: es un número de tres dígitos que no empiece por 0.
2. Departamento: en blanco o número de dos dígitos.
3. Oficio: Analista, Diseñador, Programador o Elige oficio.

DISEÑO DE PRUEBAS UNITARIAS (CLASES DE EQUIVALENCIA)

Ejercicio

Si la entrada es correcta, se asignará un salario de acuerdo a las siguientes normas:

si el Oficio es Analista → 2500

si el Oficio es Diseñador → 1500

si el Oficio es Programador → 2000

DISEÑO DE PRUEBAS UNITARIAS (CLASES DE EQUIVALENCIA)

Resolución

Condición entrada	Tipo clases de equivalencia	Clases válidas	Código	Clases no válidas	Código
Empleado es un número de tres dígitos que no empiece por 0.	Rango $999 \geq X \geq 100$	- 512	V1	- 50 (debajo) - 1200 (encima)	NV1
Departamento: en blanco o número de dos dígitos.	Valor específico	- 02	V2	- 100	NV2
Oficio: Analista, Diseñador, Programador o Elige oficio.	Miembro del conjunto	Analista	V3	- Full Stack	NV3
		Diseñador	V4		
		Programador	V5		
		Elige oficio	V6		

DISEÑO DE PRUEBAS UNITARIAS (CLASES DE EQUIVALENCIA)

Caso de Prueba	Clases de equivalencia	Condiciones de entrada		Resultado esperado
		Tantos valores como clases de equivalencia	...	
CI	VI,V2	N	M	Z

DISEÑO DE PRUEBAS UNITARIAS (CLASES DE EQUIVALENCIA)

Ejercicio

Se van a diseñar las pruebas necesarias para la entrada de datos de un empleado por pantalla gráfica. Se definen tres campos de entrada, uno de ellos es una lista desplegable para elegir oficio.

1. Empleado: es un número de tres dígitos que no empiece por 0.
2. Departamento: en blanco o número de dos dígitos.
3. Oficio: Analista, Diseñador, Programador o Elige oficio.

HERRAMIENTAS DE DEPURACIÓN

Cada IDE incluye herramientas de depuración como:

- Inclusión de puntos de ruptura
 - Ejecución paso a paso de cada instrucción
 - Ejecución por procedimiento
 - Inspección de variables, etc.
-
- Nos ayudan a encontrar errores lógicos, o bugs, que aun compilando correctamente el programa, provocan que el programa no funcione según lo esperado.
 - Para ello, en los IDE → **depurador** o *debugger*

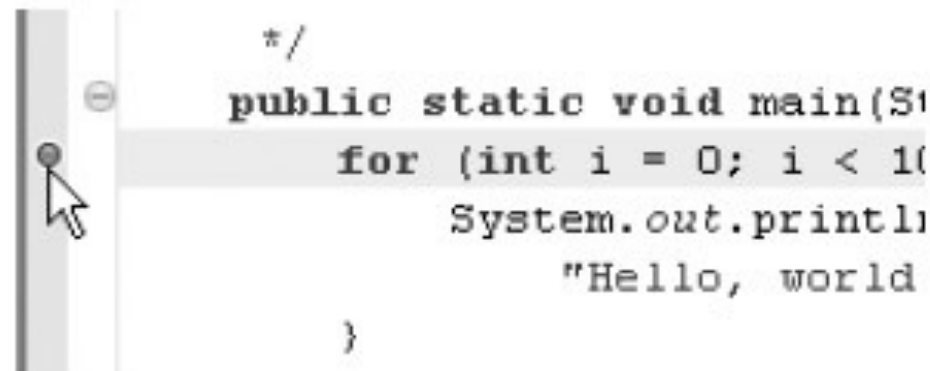
HERRAMIENTAS DE DEPURACIÓN

Debugger / depurador:

- Un programa debe compilarse con éxito para poder utilizarlo en el depurador.
- El depurador nos permite analizar todo el programa, mientras éste se ejecuta.
- Permiten:
 - Suspender la ejecución de un programa
 - Examinar y establecer los valores de las variables
 - Comprobar los valores devueltos por un determinado método
 - Comprobar el resultado de una comparación lógica o relacional
 - Etc.

DEPURACIÓN. PUNTOS DE RUPTURA

- Dentro de una aplicación, se pueden insertar varios puntos de ruptura, y se pueden eliminar con la misma facilidad con la que se insertan.
- Los puntos de ruptura son marcadores que pueden establecerse en cualquier línea de código ejecutable.



DEPURACIÓN. FUNCIONALIDADES

- Una vez insertado el punto de ruptura, e iniciada la depuración, el programa a evaluar se ejecutaría hasta la línea marcada con el punto de ruptura.
- En ese momento, se pueden realizar diferentes labores:
 - Se pueden **examinar las variables**
 - Comprobar que los **valores** que tienen asignados son correctos
 - Se pueden iniciar una **depuración paso a paso**, e ir comprobando el camino que toma el programa a partir del punto de ruptura.

DEPURACIÓN. FUNCIONALIDADES

The screenshot displays an IDE interface during a Java debugging session. The top-left pane shows the 'Debug' window with the following structure:

- DecimalToBinary [Java Application]
 - utilidades.test.DecimalToBinary at localhost:55440
 - Thread [main] (Suspended (breakpoint at line 28 in DecimalToBinary))
 - DecimalToBinary.main(String[]) line: 28

The top-right pane shows the 'Variables' window with the following table:

Name	Value
args	String
N	26

The bottom pane shows the source code of the file 'DecimalToBin...'. The code is as follows:

```
10  
19     for (int i =0; i<num.length();i++){  
20         System.out.print(num.charAt(num.length()-1-i));  
21     }  
22  
23 }  
24  
25 public static void main(String[] args) {  
26     // TODO Auto-generated method stub  
27     int N = Teclado.LeeEntero();  
28     convert(N);  
29     System.out.println();  
30     convert2(N);  
31 }  
32
```

Line 28, `convert(N);`, is highlighted in green, indicating the current execution point.

DEPURACIÓN.TIPOS DE EJECUCIÓN

- Para poder depurar un programa, podemos ejecutar el programa de diferentes formas:
 - A. Algunas veces es necesario ejecutar un programa línea por línea, para buscar y corregir errores lógicos. El **avance paso a paso** a lo largo de una parte del programa puede ayudarnos a verificar que el código de un método se ejecute en forma correcta.
 - B. El **paso a paso por procedimientos**, nos permite introducir los parámetro que queremos a un método o función de nuestro programa, pero en vez de ejecutar instrucción por instrucción ese método, nos devuelve su resultado. Es útil, cuando hemos comprobado que un procedimiento funciona correctamente, y no nos interese volver a depurarlo, sólo nos interesa el valor que devuelve.

DEPURACIÓN.TIPOS DE EJECUCIÓN

- C. En la **ejecución hasta una instrucción**, el depurador ejecuta el programa, y se detiene en la instrucción donde se encuentra el cursor, a partir de ese punto, podemos hacer una depuración paso a paso o por procedimiento.
- D. En la **ejecución de un programa hasta el final** del programa, ejecutamos las instrucciones de un programa hasta el final, sin detenernos en las instrucciones intermedias.

PRUEBAS UNITARIAS

- Las pruebas unitarias, tienen por objetivo probar el correcto funcionamiento de una unidad funcional.
- El fin que se persigue, es que cada unidad funcional sea correcta por separado.
- En programación orientada a objetos, una unidad funcional es normalmente un **método**.

PRUEBAS UNITARIAS

- En el diseño de los casos de pruebas unitarias, habrá que tener en cuenta los siguientes requisitos:
 - **Automatizable:** no debería requerirse una intervención manual.
 - **Completas:** deben cubrir la mayor cantidad de código.
 - **Repetibles o Reutilizables:** no se deben crear pruebas que sólo puedan ser ejecutadas una sola vez.
 - **Independientes:** la ejecución de una prueba no debe afectar a la ejecución de otra.
 - **Profesionales:** las pruebas deben ser consideradas igual que el código, con la misma profesionalidad, documentación, etc.

PRUEBAS UNITARIAS

- Herramientas para Java: Jtiger, TestNG, **Junit**, etc.
- **JUnit** es una herramienta de automatización de pruebas que nos permite de manera rápida y sencilla, elaborar pruebas.
 - La herramienta nos permite diseñar clases de prueba, para cada clase diseñada en nuestra aplicación.
 - Una vez creada las clases de prueba, establecemos los métodos que queremos probar, y para ello diseñamos casos de prueba.
 - Una vez diseñados los casos de prueba, pasamos a probar la aplicación.
 - La herramienta de automatización nos presentará un informe con los resultados de la prueba.
 - En función de los resultados, deberemos o no, modificar el código.

COBERTURA DE CÓDIGO

El análisis de la cobertura de código es una de las tareas que ayudan al desarrollador a saber el porcentaje del código probado con los tests.

